

■ PLAYWRIGHT

Advanced Study Guide — Part 2

*GetBy Methods · Element Interactions · Auto-Waits · Assertions
Mouse/Keyboard · Dialogs · iFrames · DDT · Page Object Model*

Chapters 9 – 18 | Complete with Code Examples

■ GetBy Methods

■ Auto-Waits

■ Mouse/KB

■ POM Pattern

■ Table of Contents

Chapter 9 — GetBy Methods & Element Interactions

- › All getBy methods
- › Input actions
- › Element controls
- › Disabled & multiple elements

Chapter 10 — Auto-Waits & Flaky Tests

- › Understanding auto-waits
- › Flaky tests explained
- › Synchronization strategies
- › Explicit waits

Chapter 11 — Assertions with expect()

- › Built-in assertions
- › Soft assertions
- › Custom validations
- › Working scripts

Chapter 12 — Mouse, Keyboard, Drag & Scroll

- › Mouse actions
- › Keyboard actions
- › Drag and Drop
- › Scroll — multiple approaches

Chapter 13 — Form Controls

- › Checkboxes
- › Radio buttons
- › All dropdown types
- › Auto-suggestions

Chapter 14 — Tabs, Windows & iFrames

- › Multiple tabs & windows
- › iFrames
- › Nested frames

Chapter 15 — Dialogs & Popups

- › Alert dialogs

- › Confirm dialogs
- › Prompt dialogs
- › Notification popups

Chapter 16 — File Upload & Download

- › Upload single & multiple files
- › Download files
- › Validating downloaded files

Chapter 17 — Data-Driven Testing (DDT)

- › Introduction to DDT
- › JSON & Excel integration
- › Single & multiple datasets
- › Parameterisation

Chapter 18 — Page Object Model (POM)

- › Introduction to POM
- › POM implementation
- › End-to-end scenario

GetBy Methods & Element Interactions

All getBy Methods — Complete Reference

getByRole()	Find by ARIA role. Most recommended for accessible elements.
getByLabel()	Find input by its associated <label> text.
getByPlaceholder()	Find input by its placeholder attribute.
getByText()	Find element by visible text content (exact or regex).
getByAltText()	Find image by its alt attribute.
getTitle()	Find element by its title attribute tooltip.
getByTestId()	Find by data-testid attribute. Most stable for automation.
locator()	Generic: CSS selector, XPath, or Playwright pseudo-selectors.
filter()	Narrow down locator results by text or another locator.
nth()	Pick the nth element from matching results (0-indexed).
first() / last()	Pick the first or last element from results.

```
// getByRole – buttons, links, headings, inputs
page.getByRole('button', { name: 'Login' })
page.getByRole('link', { name: /home/i })
page.getByRole('textbox',{ name: 'Username' })
page.getByRole('checkbox',{ name: 'Remember me' })
page.getByRole('combobox') // dropdown
page.getByRole('heading', { level: 1 })

// getByLabel – form fields with labels
page.getByLabel('Email address')
page.getByLabel(/password/i)

// getByText – any element with matching text
page.getByText('Welcome back')
page.getByText(/error/i)
page.getByText('Exact text', { exact: true })

// getByTestId – needs data-testid='...' in HTML
page.getByTestId('submit-btn')
page.getByTestId('user-avatar')

// Chaining & filtering
page.getByRole('listitem').filter({ hasText: 'Product A' })
page.getByRole('listitem').nth(2)
page.getByRole('listitem').first()
page.getByRole('listitem').last()

// Locator inside another locator
const row = page.getByRole('row', { name: 'John' });
const editBtn = row.getByRole('button', { name: 'Edit' });
await editBtn.click();
```

Basic Input Actions

```
const input = page.getByLabel('Email');

// Type text (clears existing value first)
await input.fill('user@example.com');

// Type character by character (simulates real typing)
await input.pressSequentially('Hello', { delay: 80 });

// Clear the field
await input.clear();

// Press a key while focused on element
await input.press('Enter');
await input.press('Control+A');
await input.press('Tab');

// Get current value
const val = await input.inputValue();
console.log(val);

// Focus and blur
await input.focus();
await input.blur();

// Tap (mobile)
await input.tap();
```

Element Controls — Reading State

```
const btn = page.getByRole('button', { name: 'Submit' });

// Check visibility
const isVisible = await btn.isVisible();
const isHidden = await btn.isHidden();

// Check enabled/disabled state
const isEnabled = await btn.isEnabled();
const isDisabled = await btn.isDisabled();

// Check if checked (checkbox/radio)
const isChecked = await page.getByRole('checkbox').isChecked();

// Get text content
const text = await btn.textContent();
const inner= await btn.innerHTML();

// Get attribute value
const href = await page.getByRole('link').getAttribute('href');
const cls = await btn.getAttribute('class');

// Get bounding box (position & size)
const box = await btn.boundingBox();
console.log(box); // { x, y, width, height }

// Count matching elements
const count = await page.getByRole('listitem').count();
```

Handling Disabled Elements

Disabled elements cannot be interacted with normally. Use these strategies:

```
// Check if disabled before acting
const btn = page.getByRole('button', { name: 'Submit' });
if (await btn.isEnabled()) {
  await btn.click();
} else {
  console.log('Button is disabled, skipping...');
}

// Assert it IS disabled
await expect(btn).toBeDisabled();

// Assert it IS enabled
await expect(btn).toBeEnabled();

// Force-click (bypasses actionability checks – use carefully!)
await btn.click({ force: true });

// Enable via JS evaluation (for testing purposes)
await page.evaluate(() => {
  document.querySelector('#submit').removeAttribute('disabled');
});
```

Handling Multiple Elements

```
// Get all matching elements as a list
const items = page.getByRole('listitem');

// Count them
const count = await items.count();

// Iterate over all elements
for (let i = 0; i < count; i++) {
  const text = await items.nth(i).textContent();
  console.log(`Item ${i}: ${text}`);
}

// Using allTextContents() – gets all text at once
const allTexts = await items.allTextContents();
console.log(allTexts); // ['Item 1', 'Item 2', ...]

// Using evaluateAll – runs JS on all matched elements
const values = await page.getByRole('option').evaluateAll(
  els => els.map(el => el.value)
);

// Filter to find specific item
const target = items.filter({ hasText: 'Product A' });
await target.click();

// Assert count
await expect(items).toHaveCount(5);
```

Auto-Waits & Flaky Tests

Understanding Auto-Waits in Playwright

One of Playwright's most powerful features is **automatic waiting**. Before performing any action (click, fill, etc.), Playwright automatically waits for the element to satisfy a set of **actionability checks**.

Visible	Element must be visible and not hidden by CSS.
Stable	Element must not be animating or moving.
Receives Events	Element must not be obscured by another element.
Enabled	Element must not be disabled (for inputs and buttons).
Editable	Element must not be read-only (for fill operations).

```
// Playwright automatically waits – no explicit sleeps needed!

// This waits for button to be visible + enabled + stable before clicking
await page.getByRole('button', { name: 'Submit' }).click();

// This waits for input to be visible + enabled + editable before typing
await page.getByLabel('Email').fill('user@test.com');

// Default timeout: 30,000ms (configurable in playwright.config.ts)
// use: { actionTimeout: 10000 }

// Override timeout for a single action
await page.getByRole('button').click({ timeout: 5000 });
```

Understanding Flaky Tests

Flaky tests are tests that sometimes pass and sometimes fail without any code change. They are the #1 enemy of reliable test automation. Common causes:

- **Timing issues** — Test acts before element is ready
- **Network latency** — API responses take variable time
- **Animation** — Clicking an element mid-animation

- **Race conditions** — Multiple async operations completing in unpredictable order
- **Environment differences** — Slower CI machines vs fast local machine
- **Dynamic content** — Content loaded lazily or out of order
- **Hard-coded sleeps** — `await page.waitForTimeout(2000)` causes both slowness AND flakiness

Never use `page.waitForTimeout()` in production tests. It makes tests slow AND fragile.

Synchronization Strategies

```
// Strategy 1: Wait for specific URL
await page.waitForURL('**/dashboard');
await page.waitForURL(/login/);

// Strategy 2: Wait for load state
await page.waitForLoadState('domcontentloaded'); // DOM ready
await page.waitForLoadState('load'); // all resources loaded
await page.waitForLoadState('networkidle'); // no network requests for 500ms

// Strategy 3: Wait for element to appear
await page.getByText('Welcome').waitFor({ state: 'visible' });
await page.getByText('Loading...').waitFor({ state: 'hidden' });
await page.locator('.spinner').waitFor({ state: 'detached' });

// Strategy 4: Wait for network response
const [response] = await Promise.all([
  page.waitForResponse('**/api/users'),
  page.getByRole('button', { name: 'Load Users' }).click(),
]);
console.log(await response.json());

// Strategy 5: Wait for request
const [request] = await Promise.all([
  page.waitForRequest('**/api/submit'),
  page.getByRole('button', { name: 'Submit' }).click(),
]);

// Strategy 6: Wait for function to return true
await page.waitForFunction(() => {
  return document.querySelector('.items').children.length > 0;
});
```

Explicit Waits

```
// Wait with custom timeout and state
await page.locator('#result').waitFor({
  state: 'visible', // 'visible' | 'hidden' | 'attached' | 'detached'
  timeout: 10000, // 10 seconds
});

// Poll until condition is true
await expect.poll(async () => {
  const count = await page.getByRole('listitem').count();
  return count;
}, { timeout: 15000 }).toBeGreaterThan(0);

// Retry assertion automatically
await expect(page.getByText('Success')).toBeVisible({ timeout: 8000 });

// Wait for console message
page.on('console', msg => console.log(msg.text()));

// Only use this for debugging – not in production tests!
// await page.waitForTimeout(1000);
```

Playwright's expect() assertions automatically retry until the condition is met or the timeout expires — use assertions instead of explicit waits whenever possible!

Assertions with expect()

Built-in Assertions — Complete Reference

toBeVisible()	Element is visible on screen
toBeHidden()	Element is not visible
toBeEnabled()	Element is enabled
toBeDisabled()	Element is disabled
toBeChecked()	Checkbox or radio is checked
toBeEmpty()	Input has no value, element has no text
toHaveText(str/regex)	Element contains specified text
toHaveValue(str)	Input has specified value
toHaveCount(n)	Locator matches exactly n elements
toHaveAttribute(k,v)	Element has attribute with value
toHaveClass(str)	Element has the specified CSS class
toHaveCSS(prop,val)	Element has specified CSS property
toHaveTitle(str/regex)	Page title matches
toHaveURL(str/regex)	Current URL matches
toContainText(str)	Element contains substring
toBeInViewport()	Element is in the visible viewport
toBeFocused()	Element has focus
toBeOK()	Response status is 2xx (for API testing)

Soft Assertions

By default, a failing assertion stops the test immediately. **Soft assertions** collect all failures and report them at the end — the test continues even after a failure.

```
test('soft assertion example', async ({ page }) => {
  await page.goto('https://example.com');

  // Soft assertions – test continues even on failure
  await expect.soft(page).toHaveTitle('Example Domain');
  await expect.soft(page.getByRole('heading')).toHaveText('Example Domain');
  await expect.soft(page.getByRole('link')).toBeVisible();

  // All failures are reported together at end of test
  // Hard assertion – stops test if fails
  await expect(page).toHaveURL('https://example.com/');
});
```

Custom Validations

```
import { test, expect } from '@playwright/test';

// Custom assertion using expect.extend()
expect.extend({
  async toBeWithinRange(received: number, floor: number, ceiling: number) {
    const pass = received >= floor && received <= ceiling;
    return {
      pass,
      message: () =>
        pass
        ? `Expected ${received} NOT to be in [${floor}, ${ceiling}]`
        : `Expected ${received} to be in [${floor}, ${ceiling}]`,
    };
  },
});

test('custom validation', async ({ page }) => {
  const priceText = await page.getByTestId('price').textContent();
  const price = parseFloat(priceText!.replace('$', ''));

  // Use custom assertion
  expect(price).toBeWithinRange(10, 100);

  // Custom validation with page.evaluate()
  const isValid = await page.evaluate(() => {
    const email = document.querySelector('#email') as HTMLInputElement;
    return email.validity.valid;
  });
  expect(isValid).toBe(true);

  // Validate response data
  const response = await page.request.get('/api/user/1');
  const data = await response.json();
  expect(data).toHaveProperty('name');
  expect(data.email).toMatch(/@/);
  expect(Array.isArray(data.roles)).toBe(true);
});
```

Mouse, Keyboard, Drag & Scroll

Mouse Actions

```
// Basic clicks
await page.mouse.click(100, 200);
await page.mouse.dblclick(100, 200);

// Click with modifiers
await page.getByText('Item').click({ modifiers: ['Shift'] }); // Shift+click
await page.getByText('Item').click({ modifiers: ['Control'] }); // Ctrl+click
await page.getByText('Item').click({ button: 'right' }); // Right-click

// Mouse movement
await page.mouse.move(300, 400);
await page.mouse.move(300, 400, { steps: 10 }); // smooth movement

// Mouse down/up (for drag simulation)
await page.mouse.down();
await page.mouse.move(500, 400, { steps: 20 });
await page.mouse.up();

// Hover over element
await page.getByRole('button').hover();
await page.mouse.move(150, 250); // hover by coordinates

// Click at specific position within element
await page.getByRole('button').click({ position: { x: 10, y: 5 } });
```

Keyboard Actions

```
// Press single key
await page.keyboard.press('Enter');
await page.keyboard.press('Escape');
await page.keyboard.press('Tab');
await page.keyboard.press('ArrowDown');
await page.keyboard.press('F5');

// Key combinations
await page.keyboard.press('Control+A'); // Select all
await page.keyboard.press('Control+C'); // Copy
await page.keyboard.press('Control+V'); // Paste
await page.keyboard.press('Control+Z'); // Undo
await page.keyboard.press('Shift+Tab'); // Reverse Tab
await page.keyboard.press('Meta+A'); // Cmd+A on Mac

// Type text
await page.keyboard.type('Hello World');
await page.keyboard.type('Hello', { delay: 100 }); // slow typing

// Hold and release keys
await page.keyboard.down('Shift');
await page.keyboard.press('ArrowRight'); // select char
await page.keyboard.up('Shift');

// Press on a specific element
await page.getByLabel('Search').press('Enter');
```

Drag and Drop

```
// Approach 1: dragTo() – simplest method
const source = page.locator('#drag-source');
const target = page.locator('#drop-target');
await source.dragTo(target);

// Approach 2: dragTo with offset
await source.dragTo(target, {
  sourcePosition: { x: 10, y: 10 },
  targetPosition: { x: 20, y: 20 },
});

// Approach 3: Using mouse events (for complex D&D libraries)
const sourceBox = await source.boundingBox();
const targetBox = await target.boundingBox();

await page.mouse.move(
  sourceBox!.x + sourceBox!.width / 2,
  sourceBox!.y + sourceBox!.height / 2
);
await page.mouse.down();
// Move in small steps for smooth drag
await page.mouse.move(
  targetBox!.x + targetBox!.width / 2,
  targetBox!.y + targetBox!.height / 2,
  { steps: 15 }
);
await page.mouse.up();

// Approach 4: dispatchEvent for HTML5 drag
await page.dispatchEvent('#draggable', 'dragstart');
await page.dispatchEvent('#droptarget', 'drop');
```

Scroll — Multiple Approaches

```
// Approach 1: scroll by pixels using evaluate()
await page.evaluate(() => window.scrollTo(0, 500));
await page.evaluate(() => window.scrollBy(0, 300)); // relative scroll

// Approach 2: scroll to bottom of page
await page.evaluate(() => window.scrollTo(0, document.body.scrollHeight));

// Approach 3: scrollIntoViewIfNeeded – scroll to element
await page.getByText('Footer content').scrollIntoViewIfNeeded();

// Approach 4: mouse wheel scroll
await page.mouse.wheel(0, 500); // scroll down 500px
await page.mouse.wheel(0, -300); // scroll up 300px

// Approach 5: scroll inside a scrollable container
const container = page.locator('.scroll-container');
await container.evaluate(el => el.scrollTo(0, el.scrollHeight));

// Approach 6: keyboard scroll
await page.keyboard.press('PageDown');
await page.keyboard.press('PageUp');
await page.keyboard.press('End'); // scroll to bottom
await page.keyboard.press('Home'); // scroll to top

// Approach 7: focus element and scroll with Tab
await page.getByRole('link', { name: 'Contact' }).focus();
```

Form Controls: Checkbox, Radio, Dropdowns, Auto-suggestions

Handling Checkboxes

```
// Check a checkbox
await page.getByLabel('I agree to terms').check();
await page.getByRole('checkbox', { name: 'Subscribe' }).check();

// Uncheck a checkbox
await page.getByLabel('Newsletter').uncheck();

// Check only if not already checked
const cb = page.getByLabel('Remember me');
if (!(await cb.isChecked())) {
  await cb.check();
}

// Assert checkbox state
await expect(page.getByLabel('Terms')).toBeChecked();
await expect(page.getByLabel('Newsletter')).not.toBeChecked();

// Handle multiple checkboxes
const checkboxes = page.getByRole('checkbox');
const count = await checkboxes.count();
for (let i = 0; i < count; i++) {
  await checkboxes.nth(i).check();
}
```

Handling Radio Buttons

```
// Select radio button by label
await page.getByLabel('Male').check();
await page.getByLabel('Female').check();

// Select by role + name
await page.getByRole('radio', { name: 'Option A' }).check();

// Select by value attribute
await page.locator('input[type="radio"][value="medium"]').check();

// Assert selection
await expect(page.getByLabel('Male')).toBeChecked();

// Get selected radio value using evaluate()
const selectedValue = await page.evaluate(() => {
const selected = document.querySelector('input[type="radio"]:checked');
return selected ? (selected as HTMLInputElement).value : null;
});
console.log('Selected:', selectedValue);
```

Dropdowns — All Types

1. Native HTML select element

```
// Select by visible text
await page.getByLabel('Country').selectOption({ label: 'India' });

// Select by value attribute
await page.getByLabel('Country').selectOption({ value: 'IN' });

// Select by index (0-based)
await page.getByLabel('Country').selectOption({ index: 2 });

// Multiple selection (multi-select dropdown)
await page.getByLabel('Languages').selectOption(['Python', 'Java', 'JS']);

// Deselect all options
await page.getByLabel('Languages').selectOption([]);

// Assert selected value
await expect(page.getByLabel('Country')).toHaveValue('IN');
```

2. Custom Dropdown (div/ul based)

```
// Step 1: Click dropdown trigger
await page.getByRole('button', { name: 'Select Country' }).click();

// Step 2: Wait for options to appear
await page.getByRole('listbox').waitFor({ state: 'visible' });

// Step 3: Click desired option
await page.getByRole('option', { name: 'India' }).click();

// Or using text
await page.getByText('India').click();

// Assert selection
await expect(page.getByTestId('selected-value')).toHaveText('India');
```

3. React-Select / Material UI Dropdown

```
// React-Select specific approach
const dropdown = page.locator('.react-select__control');

// Click to open
await dropdown.click();

// Type to filter options
await page.locator('.react-select__input input').fill('Ind');

// Select from filtered options
await page.locator('.react-select__option').filter({ hasText: 'India' }).click();

// Verify selection
await expect(page.locator('.react-select__single-value')).toHaveText('India');
```

Handling Auto-suggestions

```
test('auto-suggestion handling', async ({ page }) => {
  await page.goto('https://example.com/search');

  // Type in search box to trigger suggestions
  const searchBox = page.getByRole('combobox', { name: 'Search' });
  await searchBox.fill('Play');

  // Wait for suggestion list to appear
  const suggestions = page.getByRole('listbox');
  await suggestions.waitFor({ state: 'visible' });

  // Get all suggestion texts
  const allSuggestions = await page.getByRole('option').allTextContents();
  console.log('Suggestions:', allSuggestions);

  // Click specific suggestion
  await page.getByRole('option', { name: 'Playwright' }).click();

  // OR: click the first suggestion
  await page.getByRole('option').first().click();

  // Verify the selection filled the input
  await expect(searchBox).toHaveValue('Playwright');
});
```

Multiple Tabs, Windows & iFrames

Handling Multiple Tabs and Windows

When a click opens a new tab or window, Playwright provides the `context.waitForEvent('page')` method to capture the new page.

```
test('handle new tab', async ({ context, page }) => {
  await page.goto('https://example.com');

  // Method 1: waitForEvent – recommended
  const [newPage] = await Promise.all([
    context.waitForEvent('page'),
    page.getByRole('link', { name: 'Open in new tab' }).click(),
  ]);

  // Wait for new page to load
  await newPage.waitForLoadState('domcontentloaded');

  // Interact with new tab
  console.log('New tab URL:', newPage.url());
  await expect(newPage).toHaveTitle(/New Page/);
  await newPage.getByRole('button').click();

  // Close new tab and return to original
  await newPage.close();
  await expect(page).toHaveURL('https://example.com');
});

test('handle multiple tabs', async ({ context, page }) => {
  await page.goto('https://example.com');

  // Get all open pages
  const allPages = context.pages();
  console.log('Open tabs:', allPages.length);

  // Switch between pages
  await allPages[0].bringToFront();
  await allPages[1].bringToFront();
});
```

Handling iFrames

```

test('iframe interaction', async ({ page }) => {
  await page.goto('https://example.com/page-with-iframe');

  // Locate iframe by selector
  const frameLocator = page.frameLocator('#payment-iframe');
  // OR by name attribute
  const frameByName = page.frameLocator('iframe[name="checkout"]');
  // OR by URL
  const frameByURL = page.frameLocator('iframe[src*="payment"]');

  // Interact with elements INSIDE the iframe
  await frameLocator.getByLabel('Card Number').fill('4111111111111111');
  await frameLocator.getByLabel('Expiry').fill('12/26');
  await frameLocator.getByLabel('CVV').fill('123');
  await frameLocator.getByRole('button', { name: 'Pay' }).click();

  // Assert inside iframe
  await expect(frameLocator.getByText('Payment successful')).toBeVisible();
});

```

Nested Frames

```

test('nested frames', async ({ page }) => {
  await page.goto('https://example.com/nested-frames');

  // Chain frameLocator calls for nested frames
  const outerFrame = page.frameLocator('#outer-frame');
  const innerFrame = outerFrame.frameLocator('#inner-frame');

  // Interact with element deep inside nested frame
  await innerFrame.getByRole('button', { name: 'Click Me' }).click();

  // Using legacy frame() API (for complex scenarios)
  const frame1 = page.frame({ name: 'outer' });
  const frame2 = frame1?.childFrames()[0];
  await frame2?.fill('#input', 'test value');

  // Get all frames on the page
  const allFrames = page.frames();
  for (const frame of allFrames) {
    console.log('Frame URL:', frame.url());
  }
});

```

Dialogs & Popups

Handling Dialogs

JavaScript dialogs (alert, confirm, prompt) are handled via the **'dialog'** event.

Simple Alert Dialog

```
test('handle alert', async ({ page }) => {
  // Register handler BEFORE triggering the alert
  page.on('dialog', async dialog => {
    console.log('Dialog type:', dialog.type()); // 'alert'
    console.log('Dialog message:', dialog.message());
    await dialog.accept(); // Click OK
  });

  await page.goto('https://example.com');
  await page.getByRole('button', { name: 'Show Alert' }).click();
});

// Using waitForEvent (one-time handler)
const dialogPromise = page.waitForEvent('dialog');
await page.getByRole('button', { name: 'Alert' }).click();
const dialog = await dialogPromise;
expect(dialog.message()).toBe('Hello!');
await dialog.accept();
```

Confirm Dialog

```

test('handle confirm - accept', async ({ page }) => {
  page.on('dialog', async dialog => {
    // dialog.type() === 'confirm'
    await dialog.accept(); // Click OK / Yes
  });
  await page.getByRole('button', { name: 'Delete' }).click();
  await expect(page.getByText('Item deleted')).toBeVisible();
});

test('handle confirm - dismiss', async ({ page }) => {
  page.on('dialog', async dialog => {
    await dialog.dismiss(); // Click Cancel / No
  });
  await page.getByRole('button', { name: 'Delete' }).click();
  await expect(page.getByText('Deletion cancelled')).toBeVisible();
});

```

Prompt Dialog

```

test('handle prompt', async ({ page }) => {
  page.on('dialog', async dialog => {
    // dialog.type() === 'prompt'
    console.log('Default value:', dialog.defaultValue());
    // Accept with a value
    await dialog.accept('John Doe');
  });

  await page.getByRole('button', { name: 'Enter Name' }).click();
  await expect(page.getByTestId('greeting')).toHaveText('Hello, John Doe!');
});

// Dismiss prompt (user clicked Cancel)
page.on('dialog', async dialog => {
  await dialog.dismiss(); // returns null to the page
});

```

Notification Popups

Script Level — Mocking Browser Notifications

```

test('handle notification popup - script level', async ({ page }) => {
  // Grant notification permission before page loads
  await page.context().grantPermissions(['notifications']);

  // Mock the Notification API to capture notifications
  await page.addInitScript(() => {
    window.Notification = class {
      static permission = 'granted';
      static requestPermission = async () => 'granted';
      constructor(public title: string, public options?: any) {
        console.log('Notification:', title, options?.body);
        (window as any).__lastNotification = { title, options };
      }
    } as any;
  });

  await page.goto('https://example.com');
  await page.getByRole('button', { name: 'Send Notification' }).click();

  // Verify the notification was triggered
  const notif = await page.evaluate(() => (window as any).__lastNotification);
  expect(notif.title).toBe('New Message');
});

```

Project Level — playwright.config.ts

```

// playwright.config.ts
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    permissions: ['notifications', 'geolocation', 'camera', 'microphone'],
    geolocation: { latitude: 28.6, longitude: 77.2 }, // New Delhi
  },
});

// OR per-context in a test
test('project level permissions', async ({ browser }) => {
  const context = await browser.newContext({
    permissions: ['notifications'],
  });
  const page = await context.newPage();
  await page.goto('https://example.com');
  // Notifications already granted!
});

```

File Upload & Download

Upload Single File

```
test('upload single file', async ({ page }) => {
  await page.goto('https://example.com/upload');

  // Method 1: setInputFiles on file input element
  await page.getByLabel('Upload File').setInputFiles('path/to/file.pdf');
  // OR
  await
  page.locator('input[type="file"]').setInputFiles('./tests/fixtures/test.pdf');

  // Method 2: Using filechooser event (for custom upload buttons)
  const [fileChooser] = await Promise.all([
    page.waitForEvent('filechooser'),
    page.getByRole('button', { name: 'Choose File' }).click(),
  ]);
  await fileChooser.setFiles('path/to/document.pdf');

  // Submit the form
  await page.getByRole('button', { name: 'Upload' }).click();

  // Verify success
  await expect(page.getByText('File uploaded successfully')).toBeVisible();
});
```

Upload Multiple Files

```
test('upload multiple files', async ({ page }) => {
  await page.goto('https://example.com/upload');

  // Pass an array of file paths
  await page.locator('input[type="file"]').setInputFiles([
    './fixtures/file1.jpg',
    './fixtures/file2.png',
    './fixtures/document.pdf',
  ]);

  // Using filechooser for multiple files
  const [fileChooser] = await Promise.all([
    page.waitForEvent('filechooser'),
    page.getByRole('button', { name: 'Select Files' }).click(),
  ]);
  await fileChooser.setFiles([
    { name: 'file1.txt', mimeType: 'text/plain', buffer: Buffer.from('Hello') },
    { name: 'file2.txt', mimeType: 'text/plain', buffer: Buffer.from('World') },
  ]);

  // Verify multiple files listed
  await expect(page.getByTestId('file-count')).toHaveText('3 files selected');
});
```

Download Files

```
test('download file', async ({ page }) => {
  await page.goto('https://example.com');

  // Method 1: waitForEvent('download')
  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('link', { name: 'Download Report' }).click(),
  ]);

  // Get download info
  console.log('Filename:', download.suggestedFilename());
  console.log('URL:', download.url());

  // Save to specific path
  const path = `./downloads/${download.suggestedFilename()}`;
  await download.saveAs(path);

  // Method 2: Set downloads path in config
  // use: { acceptDownloads: true }
  // The download is saved automatically to a temp location
  const tempPath = await download.path();
  console.log('Temp path:', tempPath);
});
```

Validating Downloaded Files

```
import { test, expect } from '@playwright/test';
import * as fs from 'fs';
import * as path from 'path';

test('validate downloaded file', async ({ page }) => {
  await page.goto('https://example.com');

  const [download] = await Promise.all([
    page.waitForEvent('download'),
    page.getByRole('link', { name: 'Download CSV' }).click(),
  ]);

  const savePath = path.join('./downloads', download.suggestedFilename());
  await download.saveAs(savePath);

  // Validate 1: File exists
  expect(fs.existsSync(savePath)).toBe(true);

  // Validate 2: Correct filename
  expect(download.suggestedFilename()).toBe('report.csv');

  // Validate 3: File size
  const stats = fs.statSync(savePath);
  expect(stats.size).toBeGreaterThan(0);

  // Validate 4: File contents
  const content = fs.readFileSync(savePath, 'utf-8');
  expect(content).toContain('Name,Email,Date'); // header row
  expect(content.split('\n').length).toBeGreaterThan(1);

  // Validate 5: Check no download failure
  const failure = await download.failure();
  expect(failure).toBeNull();
});
```

Data-Driven Testing (DDT)

Introduction to DDT

Data-Driven Testing (DDT) means running the same test logic with multiple sets of input data. Instead of writing separate tests for each dataset, you define the data externally (JSON, Excel, CSV) and loop through it. This dramatically reduces code duplication.

- **Separation of concerns** — Test logic is separate from test data
- **Easy maintenance** — Add new test cases without changing code
- **Business-friendly** — Test data can be managed by non-technical stakeholders
- **Scalability** — Run hundreds of scenarios with minimal code

DDT with JSON Data

Test data file: testdata/users.json

```
{
  "users": [
    { "username": "john@test.com", "password": "pass123", "role": "admin" },
    { "username": "jane@test.com", "password": "pass456", "role": "user" },
    { "username": "bob@test.com", "password": "pass789", "role": "manager" }
  ],
  "invalidUsers": [
    { "username": "wrong@test.com", "password": "wrongpass", "error": "Invalid credentials" },
    { "username": "", "password": "", "error": "Email is required" }
  ]
}
```

Single dataset from JSON

```
import { test, expect } from '@playwright/test';
import testData from './testdata/users.json';

test('login with single JSON dataset', async ({ page }) => {
  const user = testData.users[0]; // Use first user

  await page.goto('/login');
  await page.getByLabel('Email').fill(user.username);
  await page.getByLabel('Password').fill(user.password);
  await page.getByRole('button', { name: 'Login' }).click();

  await expect(page.getByTestId('role-badge')).toHaveText(user.role);
});
```

Multiple datasets — parametrize with test.each equivalent

```
import { test, expect } from '@playwright/test';
import testData from './testdata/users.json';

// Loop through all users – creates one test per user
for (const user of testData.users) {
  test(`login as ${user.role}: ${user.username}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(user.username);
    await page.getByLabel('Password').fill(user.password);
    await page.getByRole('button', { name: 'Login' }).click();
    await expect(page.getByTestId('role-badge')).toHaveText(user.role);
  });
}

// Negative test cases from JSON
for (const invalid of testData.invalidUsers) {
  test(`login fails: ${invalid.error}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(invalid.username);
    await page.getByLabel('Password').fill(invalid.password);
    await page.getByRole('button', { name: 'Login' }).click();
    await expect(page.getByText(invalid.error)).toBeVisible();
  });
}
```

DDT with Excel Data

```

// Install xlsx: npm install xlsx
import * as XLSX from 'xlsx';
import * as path from 'path';

// Helper function to read Excel file
function readExcelData(filePath: string, sheetName: string) {
  const workbook = XLSX.readFile(path.join(__dirname, filePath));
  const worksheet = workbook.Sheets[sheetName];
  return XLSX.utils.sheet_to_json(worksheet); // array of objects
}

// Read data from Excel
const loginData = readExcelData('./testdata/testdata.xlsx', 'LoginTests');
// Result: [{ username: 'john@test.com', password: 'pass123', expected:
'Dashboard' }, ...]

// Run tests with Excel data
for (const row of loginData as any[]) {
  test(`Excel DDT: login as ${row.username}`, async ({ page }) => {
    await page.goto('/login');
    await page.getByLabel('Email').fill(row.username);
    await page.getByLabel('Password').fill(row.password);
    await page.getByRole('button', { name: 'Login' }).click();
    await expect(page.getByRole('heading')).toHaveText(row.expected);
  });
}

```

Parameterisation

```
import { test, expect } from '@playwright/test';

// Inline parameterisation
const searchTerms = ['Playwright', 'Selenium', 'Cypress', 'WebdriverIO'];

for (const term of searchTerms) {
  test(`search for: ${term}`, async ({ page }) => {
    await page.goto('https://www.example.com');
    await page.getByRole('searchbox').fill(term);
    await page.keyboard.press('Enter');
    await expect(page.getByTestId('results-count')).not.toHaveText('0 results');
  });
}

// Using environment variables for parameterisation
const ENV = process.env.TEST_ENV || 'staging';
const BASE_URLS: Record<string, string> = {
  staging: 'https://staging.example.com',
  prod: 'https://www.example.com',
  dev: 'http://localhost:3000',
};

test('environment-parameterised test', async ({ page }) => {
  await page.goto(BASE_URLS[ENV]);
  await expect(page).toHaveTitle(/Example/);
});

// Run with: TEST_ENV=prod npx playwright test
```

Page Object Model (POM)

Introduction to POM

The **Page Object Model (POM)** is a design pattern that creates an abstraction layer between your tests and the UI. Each page of your application has a corresponding class that encapsulates all the locators and actions for that page.

Maintainability	If UI changes, update only the Page Object — not every test.
Readability	Tests read like English: <code>loginPage.login('user', 'pass')</code>
Reusability	Page Objects are shared across multiple tests.
Separation	Test logic is separate from page-specific details.
Scalability	Easy to add new pages and tests as app grows.

Folder Structure for POM

```
my-playwright-project/  
  ■■■ pages/ # Page Object classes  
  ■ ■■■ LoginPage.ts  
  ■ ■■■ DashboardPage.ts  
  ■ ■■■ ProductPage.ts  
  ■ ■■■ CheckoutPage.ts  
  ■■■ tests/ # Test files  
  ■ ■■■ login.spec.ts  
  ■ ■■■ purchase.spec.ts  
  ■ ■■■ e2e.spec.ts  
  ■■■ testdata/ # Test data files  
  ■ ■■■ users.json  
  ■ ■■■ products.xlsx  
  ■■■ fixtures/ # Custom fixtures  
  ■ ■■■ pages.ts  
  ■■■ playwright.config.ts
```

POM Implementation

pages/LoginPage.ts

```
import { Page, Locator } from '@playwright/test';

export class LoginPage {
  readonly page: Page;

  // Locators defined as class properties
  readonly emailInput: Locator;
  readonly passwordInput: Locator;
  readonly loginButton: Locator;
  readonly errorMessage: Locator;
  readonly forgotPassword: Locator;

  constructor(page: Page) {
    this.page = page;
    this.emailInput = page.getByLabel('Email');
    this.passwordInput = page.getByLabel('Password');
    this.loginButton = page.getByRole('button', { name: 'Login' });
    this.errorMessage = page.getByTestId('error-msg');
    this.forgotPassword = page.getByRole('link', { name: 'Forgot password?' });
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.emailInput.fill(email);
    await this.passwordInput.fill(password);
    await this.loginButton.click();
  }

  async getErrorMessage(): Promise<string> {
    return await this.errorMessage.textContent() || '';
  }
}
```

pages/DashboardPage.ts

```
import { Page, Locator } from '@playwright/test';

export class DashboardPage {
  readonly page: Page;
  readonly welcomeMsg: Locator;
  readonly logoutBtn: Locator;
  readonly navMenu: Locator;
  readonly productsList: Locator;

  constructor(page: Page) {
    this.page = page;
    this.welcomeMsg = page.getByTestId('welcome-message');
    this.logoutBtn = page.getByRole('button', { name: 'Logout' });
    this.navMenu = page.getByRole('navigation');
    this.productsList = page.getByRole('list', { name: 'products' });
  }

  async logout() {
    await this.logoutBtn.click();
    await this.page.waitForURL('**/login');
  }

  async navigateTo(section: string) {
    await this.navMenu.getByRole('link', { name: section }).click();
  }

  async isLoading(): Promise<boolean> {
    return await this.welcomeMsg.isVisible();
  }
}
```

End-to-End Scenario Using POM

fixtures/pages.ts — Custom fixture for POM

```
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
import { DashboardPage } from '../pages/DashboardPage';

type Pages = {
  loginPage: LoginPage;
  dashboardPage: DashboardPage;
};

export const test = base.extend<Pages>({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  },
  dashboardPage: async ({ page }, use) => {
    await use(new DashboardPage(page));
  },
});

export { expect } from '@playwright/test';
```

tests/e2e.spec.ts — Successful login & invalid login tests

```
import { test, expect } from '../fixtures/pages';
import testData from '../testdata/users.json';

test.describe('E2E: Login and Dashboard Flow', () => {

  test('successful login navigates to dashboard', async ({
    loginPage, dashboardPage
  }) => {
    const user = testData.users[0];
    await loginPage.goto();
    await loginPage.login(user.username, user.password);
    expect(await dashboardPage.isLoaded()).toBe(true);
    await expect(dashboardPage.welcomeMsg).toContainText('Welcome');
  });

  test('invalid login shows error message', async ({ loginPage }) => {
    await loginPage.goto();
    await loginPage.login('wrong@email.com', 'wrongpass');
    const error = await loginPage.getErrorMessage();
    expect(error).toBe('Invalid credentials');
  });
});
```

tests/e2e.spec.ts — Full E2E flow + DDT with POM

```

test('full E2E: login, navigate, logout', async ({
  loginPage, dashboardPage
}) => {
  // Step 1: Login
  await loginPage.goto();
  await loginPage.login('admin@test.com', 'admin123');

  // Step 2: Verify dashboard is loaded
  await expect(dashboardPage.welcomeMsg).toBeVisible();

  // Step 3: Navigate to Products section
  await dashboardPage.navigateTo('Products');
  await expect(dashboardPage.page).toHaveURL('**/products');

  // Step 4: Logout and verify redirect
  await dashboardPage.logout();
  await expect(loginPage.page).toHaveURL('**/login');
  await expect(loginPage.emailInput).toBeVisible();
});

// Data-driven POM: one test per user from JSON
for (const user of testData.users) {
  test(`DDT + POM: login as ${user.role}`, async ({
    loginPage, dashboardPage
  }) => {
    await loginPage.goto();
    await loginPage.login(user.username, user.password);
    await expect(dashboardPage.welcomeMsg).toBeVisible();
    await dashboardPage.logout();
  });
}

```

POM + DDT + Custom Fixtures = the gold standard for scalable Playwright automation. This pattern is used in large enterprise projects with hundreds of test cases.